



Agentic AI Engineering Internship , 2026

WEEK 4

LangGraph, Workflows & State Machines

Monday 13 July – Friday 17 July 2026

Orchestrating complex AI flows with graph-based state management

Week Overview

Week 4 moves beyond linear chains into graph-based orchestration. Interns learn LangGraph's node-edge-state model to build agents that branch, loop, persist state across sessions, and pause for human approval, the foundation for every complex agentic system built in the remaining weeks.

Detail	Information
Week Dates	Monday 13 July – Friday 17 July 2026
Theme	LangGraph, Workflows & State Machines
Primary Stack	Python · LangGraph · LangChain · LangSmith · SqliteSaver
Total Commitment	20 hours (4 hours/day · 5 days)
Deliverable Due Friday	1 Intermediate Project + 1 Production Project (choose one of each)

Learning Objectives

- Understand graph-based agent orchestration and why it improves on simple chains
- Build LangGraph workflows with nodes, edges, and conditional routing
- Implement stateful agents with TypedDict and Annotated state management
- Use human-in-the-loop patterns for approval workflows
- Build cyclic workflows with proper termination conditions
- Debug LangGraph workflows using LangSmith tracing

Topics & Subtopics

Topic	Subtopics
LangGraph Core	StateGraph, nodes, edges, conditional edges, END node, compilation, checkpointers
State Management	TypedDict state schema, Annotated reducers, message history management, partial state updates
Graph Patterns	Sequential, branching, looping, parallel execution, subgraphs, parent/child graphs

Topic	Subtopics
Human-in-the-Loop	Interrupt patterns, human approval nodes, breakpoints, state persistence across interrupts
Streaming & Debugging	stream() vs invoke(), LangSmith tracing, step-by-step debugging, graph visualization
Persistence	MemorySaver, SqliteSaver, checkpointing, resuming interrupted workflows

Daily Breakdown

Each day is structured around a core learning block followed by applied, hands-on practice. Complete each day's practice before moving to the next , concepts compound throughout the week.

Day	Focus	Core Learning	Applied Practice
Mon	LangGraph Foundations	Read LangGraph documentation in full. Study the State → Node → Edge mental model. Build first graph: linear 3-node pipeline with typed state.	Lab: Build a document processing graph (load → validate → chunk → embed → confirm). Add conditional edge: if doc too large, split into parts first.
Tue	Branching & Loops	Study conditional edges and routing functions. Build a classification router: user query → [general/technical/sensitive] → specialized handler.	Build a self-correcting agent loop: generate → validate → [pass: respond fail: regenerate]. Loop up to 3 times on failure.
Wed	Stateful Agents	Study TypedDict state schemas and Annotated reducers. Build a multi-turn agent that accumulates: messages, tool calls, intermediate results.	Build: Research workflow graph. State includes: query, search results, draft, feedback, final report. The agent iterates on the draft until the quality threshold is met.
Thu	Human-in-the-Loop	Study LangGraph interrupt patterns. Build an approval workflow: agent proposes action → human reviews → approves/rejects → agent adjusts.	Build: Content moderation graph. Posts flow through: auto-approve → borderline human review → final decision. Persist state between human reviews.
Fri	Production Graphs	Add LangSmith tracing to all graphs. Study SqliteSaver for persistence. Build a graph that can be interrupted and resumed days later.	Standup: demo your most complex graph with live state visualization. Architecture review: when to use LangGraph vs simple chains?

Recommended Resources

These resources are drawn from official documentation and original research. Complete the highlighted readings before their corresponding daily session.

Type	Resource	Why It Matters
Docs	LangGraph Documentation (official)	Primary reference , read all sections
Docs	LangGraph Academy (free course)	Structured LangGraph curriculum
Video	LangGraph Tutorial Series (LangChain YouTube)	Video walkthrough of key patterns
Paper	FlowMind: Automatic Workflow Generation with LLMs	Theoretical basis for LLM workflows
Blog	Harrison Chase , Why We Built LangGraph	Design philosophy and motivation
Repo	LangGraph Examples Repository (GitHub)	50+ example graphs
Video	LangGraph Human-in-the-Loop Tutorial (YouTube)	HITL implementation walkthrough
Docs	LangSmith Documentation	Tracing and debugging LangGraph
Blog	Building Production AI Workflows with LangGraph (2024)	Real-world patterns

Hands-on Labs

Complete all three labs over the course of the week. Each lab builds toward the skills required for your Friday project submission.

Lab 4.1 , Document Processing Graph

Build a LangGraph workflow with a load → validate → chunk → embed → confirm pipeline. Add a conditional edge so that oversized documents are automatically split into parts before continuing.

Lab 4.2 , Self-Correcting Agent Loop

Build a generate → validate → respond/regenerate loop that retries up to 3 times when validation fails. Track and log the number of iterations required for each input.

Lab 4.3 , Human-in-the-Loop Approval Workflow

Build a content moderation graph: posts flow through auto-approve, borderline human review, and final decision stages. State must persist correctly across the human review interrupt, even if resumed later.

Weekly Assessment

Complete this self-assessment before Friday's standup. These questions test conceptual clarity, not memorisation.

Question Type	Question
Conceptual	What does a graph-based architecture give you that a linear chain cannot? Give a concrete example.
Conceptual	Explain the role of a checkpoint in LangGraph. Why is persistence important for human-in-the-loop workflows?
Conceptual	What is the difference between a conditional edge and a regular edge? When would you use each?
Technical	Write the TypedDict state schema for a graph that needs to accumulate a list of messages and a single draft string.
Technical	Explain how an interrupt pauses a LangGraph execution and how the graph later resumes from that exact point.
Design	Design a LangGraph workflow for a content moderation system with both an automatic path and a human-review path.

Weekly Standup Requirements

Every Friday standup follows the same structure. Prepare each item in advance.

Item	Detail
Demo	Live demo of your most complex graph (2 minutes). Show state visualization or LangSmith trace.
Technical Question	Explain one branching, looping, or persistence concept from the week that changed how you think.
Architecture Review	Draw your graph's full node and edge structure, including every conditional branch.
Code Review	Share your state schema (TypedDict) and explain why each field is structured the way it is.
Blockers	Name one workflow design or persistence challenge you faced. How did you solve it (or plan to)?

Week 4 Projects

Choose exactly one Intermediate project and one Production project to complete by Friday. Both are submitted via GitHub and demonstrated in the Friday standup.

Category Projects , Choose One

Project 4-I-A: Content Creation Pipeline

Content Creation Pipeline	
Difficulty	Intermediate
Description	Build a LangGraph workflow for blog post creation: research → outline → draft → review → revise → publish. Each stage is a node. Quality gate between draft and review.
Architecture	LangGraph StateGraph: research node → outline node → draft node → quality gate (conditional) → revise node → publish node. State: topic, outline, draft, feedback, final.
Skills Learned	LangGraph state management, conditional routing, iterative refinement, content generation
Deliverables	Working pipeline, 3 generated blog posts with revision histories, graph visualization
Evaluation Criteria	Pipeline produces quality content. Quality gate correctly identifies poor drafts. State transitions are clean.

Project 4-I-B: Customer Onboarding Agent

Customer Onboarding Agent	
Difficulty	Intermediate
Description	Build a multi-step onboarding workflow: collect info → validate → generate account → send welcome → schedule follow-up. Human approval required for large accounts.
Architecture	LangGraph: collect info → validate → [standard: auto approve large: human review] → create account → notify → schedule
Skills Learned	Workflow design, human-in-the-loop, conditional routing, state persistence

Deliverables	Working workflow with CLI demo, human approval simulation, state diagram
Evaluation Criteria	Workflow handles both paths correctly. Human approval interrupts and resumes cleanly. State is preserved.

Project 4-I-C: Iterative Code Generator

Iterative Code Generator	
Difficulty	Intermediate
Description	Build a LangGraph agent that generates code, runs it (safely), checks for errors, and iterates until the code works or max iterations reached.
Architecture	LangGraph Loop: generate code → execute code → check output → [success: return error: debug node → generate code (loop)]
Skills Learned	Looping graphs, code execution, error analysis, termination conditions
Deliverables	Working code generator, 10 test problems with solutions, iteration count statistics
Evaluation Criteria	Solves 70%+ of test problems. Correctly terminates on success and max iterations. Safe code execution.

Category 2 Projects , Choose One

Project 4-P-A: AI-Powered Hiring Pipeline

AI-Powered Hiring Pipeline	
Difficulty	Production
Description	Build a complete hiring workflow: ingest resumes → score against job description → shortlist → generate interview questions → human review → decision. Includes bias detection node and audit logging.
Architecture	FastAPI + LangGraph: ingest resumes → score candidates → bias check → shortlist → generate questions → human review (HITL) → decision → audit log → SQLite
Skills Learned	Complex LangGraph workflows, bias detection, HITL, audit logging, FastAPI integration, SQLite
Deliverables	Docker-deployed system, demo with 10 sample resumes, audit logs, bias detection report, GitHub repo
Evaluation Criteria	Pipeline works end-to-end. HITL works correctly. Bias detection flags suspicious patterns. Audit logs are complete.

Project 4-P-B: Autonomous Research Workflow Engine

Autonomous Research Workflow Engine	
Difficulty	Production
Description	Build a research engine that autonomously plans a research strategy, executes parallel research threads, synthesizes findings, identifies gaps, initiates additional research if gaps found, and produces a final report.
Architecture	LangGraph: plan research → parallel research (subgraph, 3 threads) → synthesize → gap analysis → [complete: generate report gaps: plan additional (loop)] → publish
Skills Learned	Parallel subgraphs, dynamic planning, gap analysis, cyclic workflows, report generation
Deliverables	Working engine, 5 research reports on complex topics, architecture diagram, LangSmith traces exported
Evaluation Criteria	Produces comprehensive reports. Gap detection works correctly. Parallel execution verified in traces.

Project 4-P-C: Production Workflow Orchestration Platform

Production Workflow Orchestration Platform	
Difficulty	Production
Description	Build a general-purpose workflow orchestration platform where workflows are defined in YAML, compiled to LangGraph graphs at runtime, executed with full persistence, and monitored via a dashboard.
Architecture	YAML Workflow Spec → LangGraph Compiler → Execution Engine (SqliteSaver) → REST API → WebSocket for real-time status → Streamlit Dashboard
Skills Learned	Dynamic graph compilation, YAML DSL design, WebSocket, persistence, real-time monitoring
Deliverables	Platform with 5 example YAML workflows, real-time dashboard, API docs, Docker deployment, portfolio blog post
Evaluation Criteria	YAML workflows compile and run correctly. Real-time status works. Dashboard shows graph visualization. Persistence survives restarts.

Week 4 Completion Checklist

Confirm every item below before Friday's standup.

	Checklist Item
☐	All 5 daily learning sessions completed (Mon–Fri)
☐	Lab 4.1 , Document processing graph built with conditional splitting logic
☐	Lab 4.2 , Self-correcting agent loop implemented with bounded retries
☐	Lab 4.3 , Human-in-the-loop approval workflow built with verified state persistence
☐	Weekly Assessment questions answered in writing
☐	One Intermediate project chosen, built, and pushed to GitHub
☐	One Production project chosen, built, and pushed to GitHub
☐	Both projects have complete README files with setup instructions
☐	Full graph diagram (nodes, edges, conditionals) prepared for Friday's standup review
☐	Demo rehearsed and ready to present in under 2 minutes